

JUICER

Juicer is a toolkit providing a collection of finishing touches to your game. These can help to 'juice' up your game, improving game feel, feedback, and visual excitement.

SPRITE SHADER	2
COMPONENTS	3
SPRITE GHOSTING	3
SQUISHY COLLISION	5
HIT FLASH	6
BOBBING	7
FREEZE FRAME	8
SQUASH AND STRETCH	9
SCREEN SHAKE	10
MOVE WOBBLE	11
PARALLAX & INFINITE SCROLLING	12

SPRITE SHADER

Juicer utilizes a sprite shader for some effects.

- It can be found here: *Juicer/Core/Shaders/Juicer_SpriteUnlit*

The shader must be used on SpriteRenderer if you wish to utilize **Hit Flash**.

The shader has three main features, as shown off in the *Juicer_Demo_Shader* scene.

- **Outline** - Renders an outline of a specified color around the sprite.
- **Distortion** - Distorts the sprite with random noise. The frequency, amplitude, and scroll speed can all be adjusted.
- **Hit Flash** - Changes the color of the sprite and can be faded in. Used by the hit flash component.

Outline



Distortion



Hit Flash



COMPONENTS

Juicer features a number of drag and drop components. If you wish to call their functions from script, simply use the **Juicer** library.

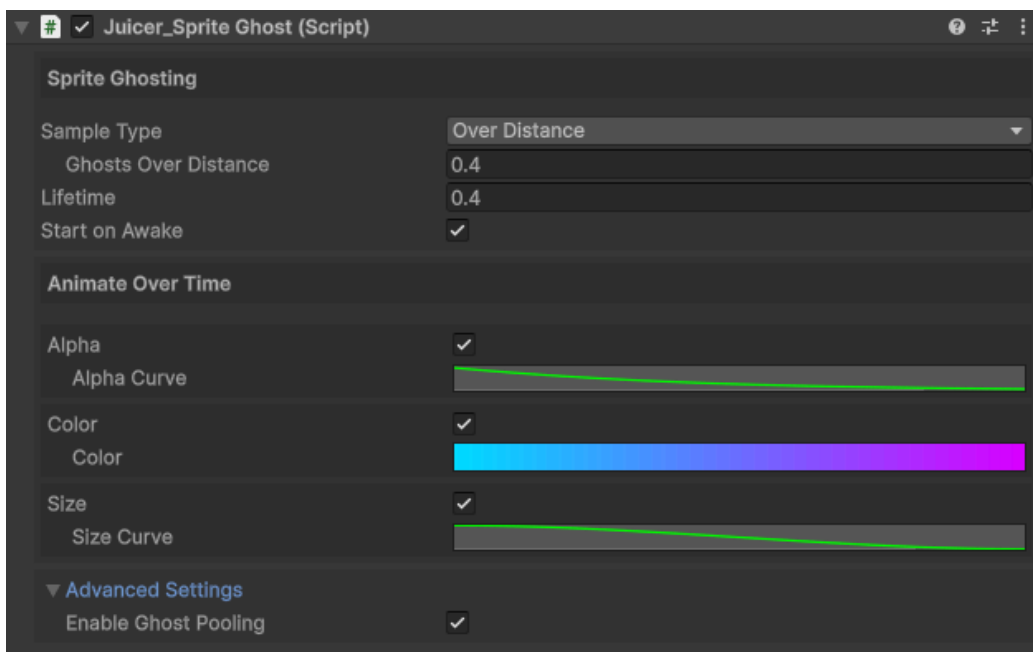
SPRITE GHOSTING

Sprite ghosting will add a trail for your sprite renderer. This will spawn copies of the sprite along its path, then applying optional animation effects.

Example implementations:

- When the player dashes to simulate speed.
- As a way of smearing fast movement.

To use sprite ghosting, add the **Juicer_SpriteGhost** component to your GameObject.



Properties

- **Sample Type** - Define how the ghost sprites will be emitted.
 - **Ghosts Over Time** - Spawn ghost every x seconds.
 - **Ghosts Over Distance** - Spawn ghost every x units moved.
- **Lifetime** - Amount of time the ghost sprite will be active for.
- **Start on Awake** - Begin the ghosting effect upon initialization.
- **Alpha Over Time** - Change the transparency of the ghost sprite over the course of its life.
 - **Alpha Curve** - Animation curve to sample from.
- **Color Over Time** - Change the color of the ghost sprite over the course of its life.

- **Color** - Gradient to sample from.
- **Size Over Time** - Change the scale of the ghost sprite over the course of its life.
 - **Size Curve** - Animation curve to sample from.
- **Enable Ghost Pooling** - Pooling will reuse old ghost sprites instead of creating new ones. This can help greatly with performance.

Functions

- **Play (float duration = 0)** - Begin the ghosting effect. Call Stop() to cease ghosting. Optionally, provide a duration to automatically stop after a certain amount of time.
- **Stop ()** - Stop the ghosting effect.
- **CreateNewGhost ()** - Manually create a new ghost.

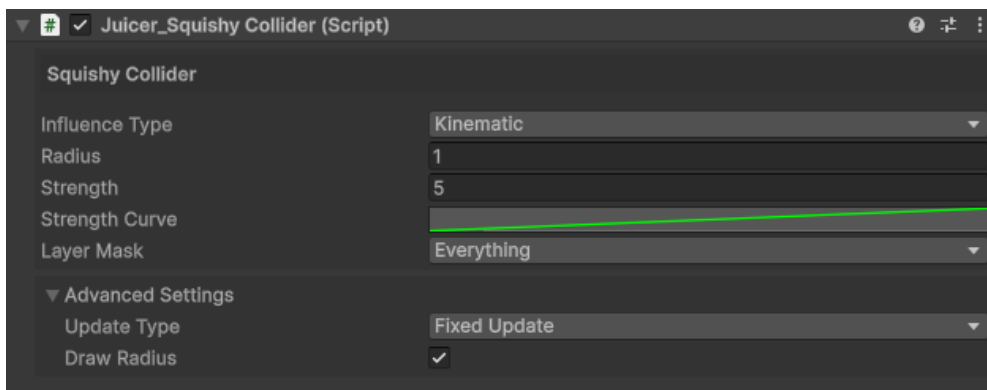
SQUISHY COLLISION

Squishy collision allows your game object to have repulsive force. This means it will be pushed away from colliders it tries to enter.

Example implementations:

- Large groups of enemies can have clumping issues mitigated by making them squishy - they will instead bump into each other, still allowing movement in between.

To use squishy collision, add the **Juicer_SquishyCollider** component to your GameObject.



Properties

- **InfluenceType** - How the calculated force will be applied.
 - *Kinematic* - Direct position translation.
 - *PhysicsAddForce* - Add force to the rigidbody.
 - *ForceOutput* - Use your own custom implementation by reading the 'PushForce' property.
- **Radius** - Radius from which the repulsion will begin.
- **Strength** - Amount of force to apply to self.
- **Strength Curve** - Rate of force depending on distance from collision.
- **Layer Mask** - Layers to check for collision.
- **Update Type** - How often squishy collision is calculated.
 - *FixedUpdate* - Using the FixedUpdate function.
 - *Update* - Using the Update function (not recommended).
 - *Custom Rate* - Using a custom update rate (recommended for best performance).
- **Draw Radius** - Draw a yellow circle (based on radius) in the scene view.

HIT FLASH

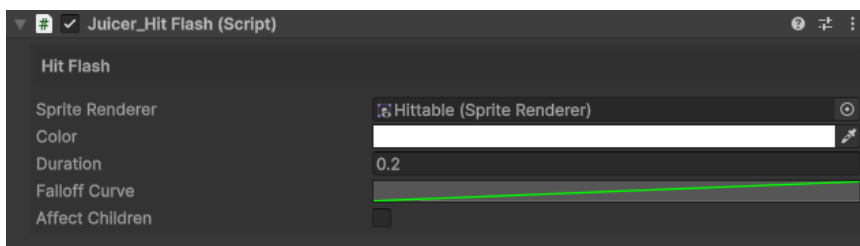
Hit flash changes the color of your sprite temporarily, then fades it out over time, or whatever falloff rate you desire. This can be used to enhance impacts.

Example implementations:

- When an enemy is hit by the player, flash them white or red.
- When the player is poisoned, flash them green whenever damage is inflicted.

To use hit flash, add the **Juicer_HitFlash** component to your GameObject.

- Make sure whatever sprites you wish to affect have a material with the *Juicer_SpriteUnlit* shader.



Properties

- **Sprite Renderer** - The SpriteRenderer component to affect.
- **Color** - The color of the flash.
- **Falloff Curve** - Change the rate at which the flash will dissipate.
- **Affect Children** - Enabling this will apply the hit effect to all children GameObjects that have a SpriteRenderer component.

Functions

- **Flash ()** - Flash the sprite using the pre-defined properties.
- **Flash (float duration)** - Flash, overriding the duration.
- **Flash (Color color)** - Flash, overriding the color.
- **Flash (Color color, float duration)** - Flash, overriding the color and duration.
- **Flash (Color color, float inDuration, float outDuration)** - Flash, overriding the color, and specifying the duration of fading in and fading out.
- **Flash (Color color, float inDuration, float holdDuration, float outDuration)** - Flash, overriding the color, and specifying the duration of fading in, how long to hold the color, then the fade out duration.

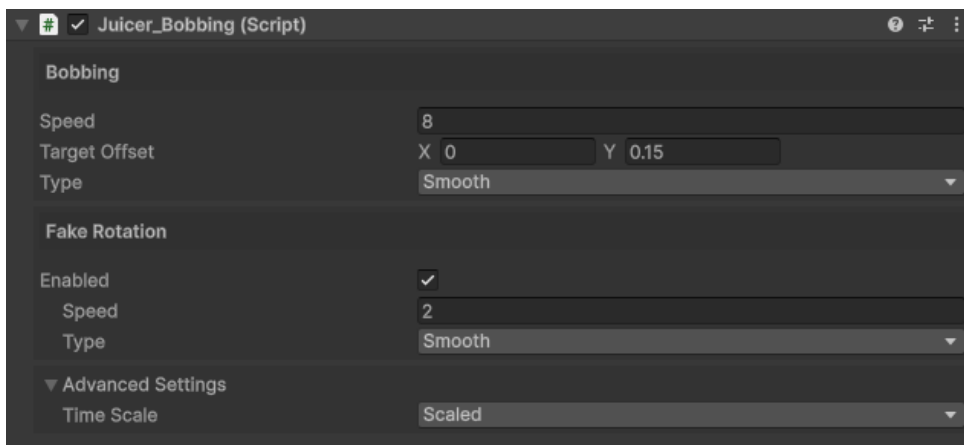
BOBBING

Bobbing can be used to make an item or collectable float up and down to attract the player's attention.

Example implementations:

- Collectable coins can bob up and down, drawing the player's attention.
- Indicator icons above enemy heads.

To use bobbing, add the **Juicer_Bobbing** component to your GameObject.



Properties

- **Bobbing**
 - **Speed** - Speed at which the object will move over time.
 - **Target Offset** - The offset from its starting position the object will move to.
 - **Type** - Changes how the object moves. Smooth: ease in and out with a sin wave. Custom: use an animation curve to determine motion over time.
 - **Custom Bob Curve** - Rate of bob motion over time. Ping pongs so it'll loop.
- **Fake Rotation**
 - **Enabled** - Apply looping to the object's X scale, simulating Y axis rotation.
 - **Speed** - Speed at which it will rotate.
 - **Type** - Changes how the rotation looks. Smooth: ease in and out with a sin wave. Linear: rotate linearly using `Mathf.PingPong()`.
- **Advanced Settings**
 - **Time Scale** - Determines whether the bobbing will run based on scaled or unscaled time. Scaled: moves based on `Time.time`. Unscaled: moves based on `Time.unscaledTime` (useful if you want the bobbing to occur when the game is paused).

FREEZE FRAME

Freeze frame can be used to emphasize impacts. It pauses the game for a specified amount of time.

Example implementations:

- When a player's attack impacts an enemy, freeze for 0.2 seconds to sell the hit.

To use freeze frame, no component is needed to be attached anywhere since it runs off a static function.

Functions

- **Trigger (float duration)** - Freezes the game for a specified amount of time. Do note that this is an async function.
 - In order to call this, you must implement the **Juicer** library.

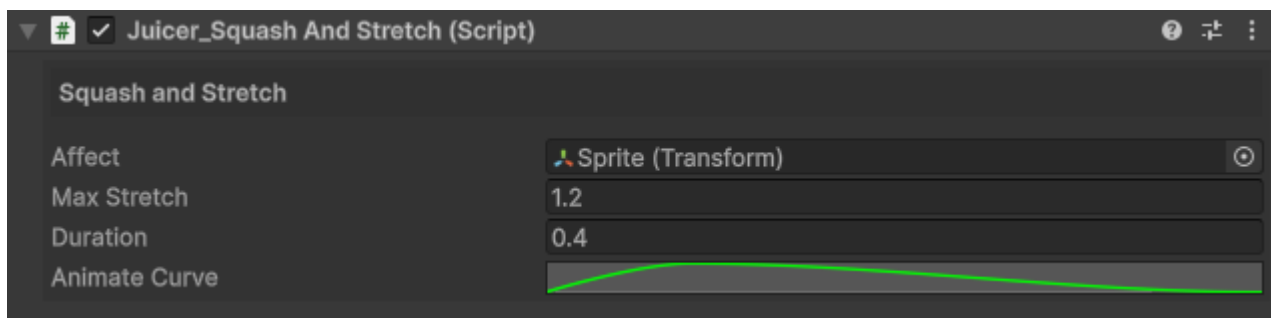
SQUASH AND STRETCH

Squash and stretch alters the scale of the object to simulate it molding to impacts and velocity change. This component can either stretch or squash the object via a function call.

Example implementations:

- When the player jumps, stretch them out, then when they land, squash them.
- When a soft object falls and hits the ground, squash it.

To use squash and stretch, add the **Juicer_SquashAndStretch** component to your GameObject.



Properties

- **Affect** - The Transform to apply the squash and stretch to.
- **Max Stretch** - Max scale offset.
- **Duration** - Total duration of the squash/stretch animation.
- **Animate Curve** - Determines the rate of change for the squash/stretch.

Functions

- **Squash ()** - Squash the transform based on the component's properties.
- **Squash (float duration, float maxStretch)** - Squash the transform and override the component's properties.
- **Stretch ()** - Stretch the transform based on the component's properties.
- **Stretch (float duration, float maxStretch)** - Stretch the transform and override the component's properties.

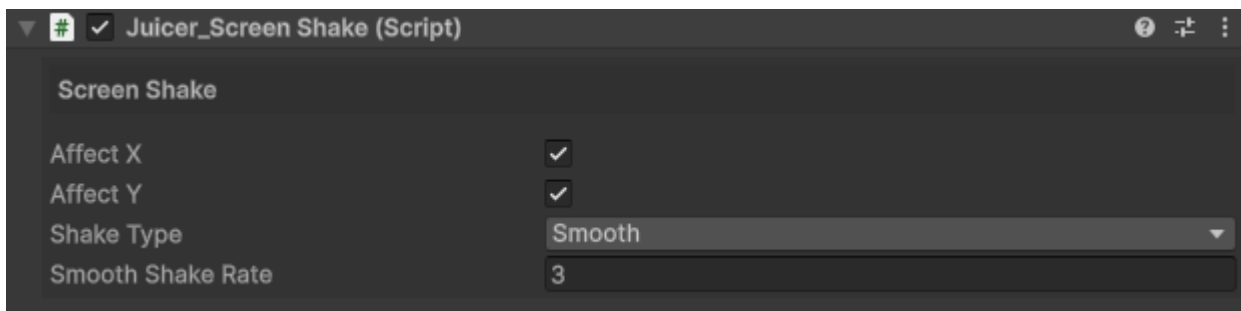
SCREEN SHAKE

Screen shake will alter the position of the camera to simulate impacts, kickback, etc.

Example implementations:

- When the player is hit, shake the screen to provide feedback.
- When a boss enters view, each footstep can trigger a screen shake.

To use screen shake, add the **Juicer_ScreenShake** component to your GameObject.



Properties

- **Affect X** - Applies shake to the X axis.
- **Affect Y** - Applies shake to the Y axis.
- **Shake Type** - Method of applying shake. Rigid: set position to a random offset each frame. Smooth: lerp between random offsets over time.
 - **Smooth Shake Rate** - Speed at which a smooth shake will move at.

Functions

- **Shake (float amount, float duration)** - Shake the camera by amount for duration seconds.
- **Shake (float amount, float duration, ShakeType shakeType)** - Shake the camera by amount for duration seconds, overriding the shake type.

If the camera moves in your scene, I recommend you have the camera as a child object of the GameObject which moves. This is because the camera shake directly modifies the camera's local position.

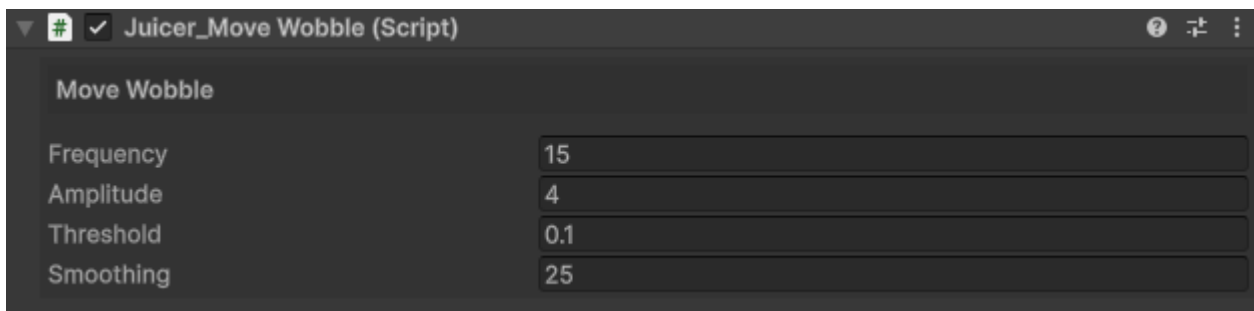
MOVE WOBBLE

Move wobble will wobble the rotation back and forth when the object is moving. This can be used as a movement animation substitute.

Example implementations:

- When the player moves, their sprite wobbles back and forth.

To use move wobble, add the **Juicer_MoveWobble** component to your GameObject.



Properties

- **Frequency** - Speed of rotation.
- **Amplitude** - Amount of rotation.
- **Threshold** - Minimum velocity required to begin wobble animation.
- **Smoothing** - Lerp strength applied to rotation. Keep this at around 25. Any lower and it will look lazier.

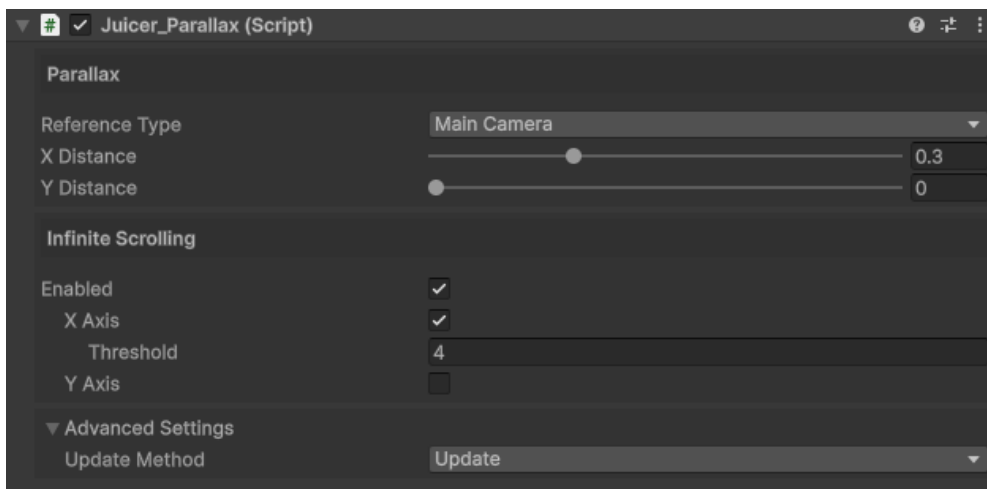
PARALLAX & INFINITE SCROLLING

Parallax allows background elements to move relative (or partially relative) to the camera, simulating depth with an orthographic camera.

Example implementations:

- You may have multiple layers (trees, mountains, sky) that have different X Distance's so they move at different rates relative to the camera, faking the illusion that the background is far away.

To use parallax & infinite scrolling, add the **Juicer_Parallax** component to your GameObject.



Properties (Parallax)

- **Reference Type** - The point of reference parallax should occur from.
 - **Reference Component** - The Transform which parallax will occur from. This will almost always be the camera.
- **X Distance** - Amount of parallax to apply to the X axis. 0 = closest, 1 = infinite distance.
- **Y Distance** - Amount of parallax to apply to the Y axis. 0 = closest, 1 = infinite distance.

Properties (Infinite Scrolling)

- **Enabled** - Teleport the object so that it continues to loop when the camera goes beyond its bounds.
 - **X Axis** - Enable infinite scrolling on the X axis.
 - **Threshold** - Distance from the camera that the object should teleport ahead on the X axis. The amount teleported is twice this amount.
 - **Y Axis** - Enable infinite scrolling on the Y axis.
 - **Threshold** - Distance from the camera that the object should teleport ahead on the Y axis. The amount teleported is twice this amount.
- **Update Method** - The method we'll use to process the parallax.
 - **Update Rate** - Rate of updating the parallax if Update Method is Custom.